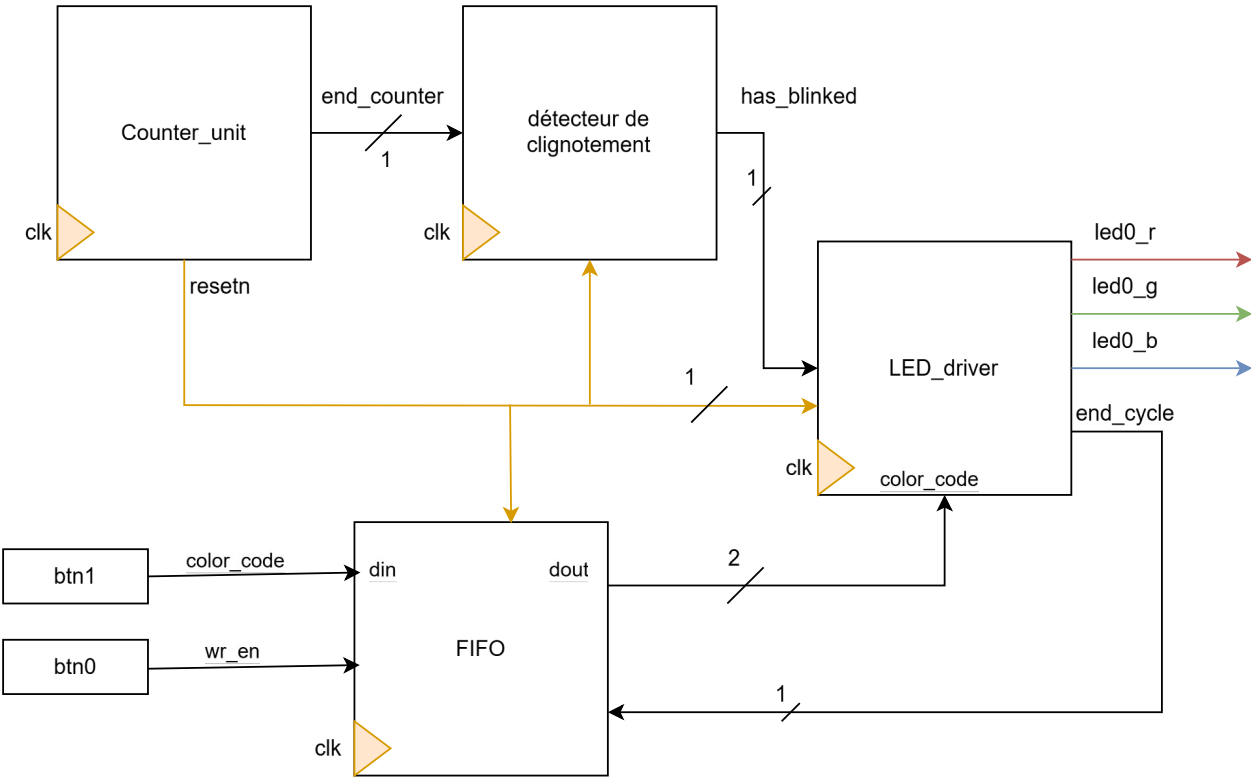


TP4 – Pilotage de LED et mémoire Partie 2

1. Sur l'architecture RTL, modifiez le module LED_driver en ajoutant une sortie end_cycle. Cette sortie vaudra 1 à la fin d'un cycle allumé/éteint de la LED RGB.

J'ai apporté quelques modifications au schéma RTL pour qu'il corresponde à cette partie 2, je l'ai complété au fur et à mesure des questions de la partie 2.

On a toujours counter_unit et le détecteur de clignotement qui servent à faire clignoter les leds. LED_driver a bien une sortie end_cycle calculé avec les signaux du détecteur de clignotement et envoyé dans la FIFO. Les boutons 0 et 1 qui transmette les signaux d'entrée de la FIFO et dout qui renvois la couleur à afficher dans le code couleur de LED_driver qui peut à son tour piloter les différente leds.



2. Modifiez la logique en entrée du module pour ajouter une FIFO. Cette FIFO doit prendre en entrée le code couleur « vert » ou « bleu » suivant l'état du bouton_1 et est connectée en sortie à l'entrée color_code du module LED_driver. La donnée est écrite dans la FIFO lorsqu'il y a un front montant du bouton_0. La donnée de la FIFO est lue lorsque le signal end_cycle du module LED_driver vaut 1.

Pour ajouter une FIFO, ouvrez le catalog d'IP de Vivado (onglet Project Manager -> IP Catalog) et recherchez l'IP qui vous semble la plus pertinente. Les documentations des IPs de Xilinx sont disponibles en ligne, cela peut faciliter vos recherches.

Une fois que vous avez choisi une IP dans le catalogue, double cliquez dessus et choisissez les paramètres. Cliquez ensuite sur OK. Pour vous aider pour l'implémentation de cette IP, vous pouvez ouvrir un design d'exemple. Pour cela, faites un clic droit sur l'IP dans l'onglet Sources, puis Open IP Example Design.

Voir code pilote_led_driver,

Qu'est-ce qu'une FIFO ?

Une FIFO (First In, First Out) est une mémoire tampon qui stocke des données dans l'ordre où elles arrivent, et les restitue dans ce même ordre, un peu comme une file d'attente. Elle possède une interface d'écriture (wr_en) et une interface de lecture (rd_en) indépendante, des données d'entrée et de sortie (din & dout), ainsi que des drapeaux d'état (full, empty).

J'ai configuré ma FIFO de la manière suivante :

Interface type : Native

Common clock

Read mode : FWFT (First-Word Fall-Through) pour accélérer la lecture de la prochaine donnée

Write width & Read width : 2 bits, correspond à la largeur de color_code

Write depth : 32, c'est la longueur de la FIFO (file d'attente)

Synchronous reset (srst)

Pour le câblage :

l'entrée	din	est piloté par le signal color_code.
le signal	wr_en	est piloté par l'ancien signal « update » qui détectait les fronts montant de btn0 que j'ai renommé « btn0_m ».
le signal	rd_en	est piloté par un signal read_en lui même piloté par « end_cycle » et « not empty ». De cette manière on ne peut lire la file d'attente que lorsqu'un cycle de clignotement vient de se terminer et qu'une donnée est présente dans la file d'attente. J'utilise un « not empty » pour protéger le fait de lire la FIFO alors qu'il n'y pas aucune valeur en file d'attente, une action qui renvoie une erreur « underflow ».
la sortie	dout	renvoie une donnée dans le port color_code et LED_driver et change la couleur de la led.

3. Modifiez vos codes de la partie 1 pour y ajouter les nouveaux éléments de votre architecture.

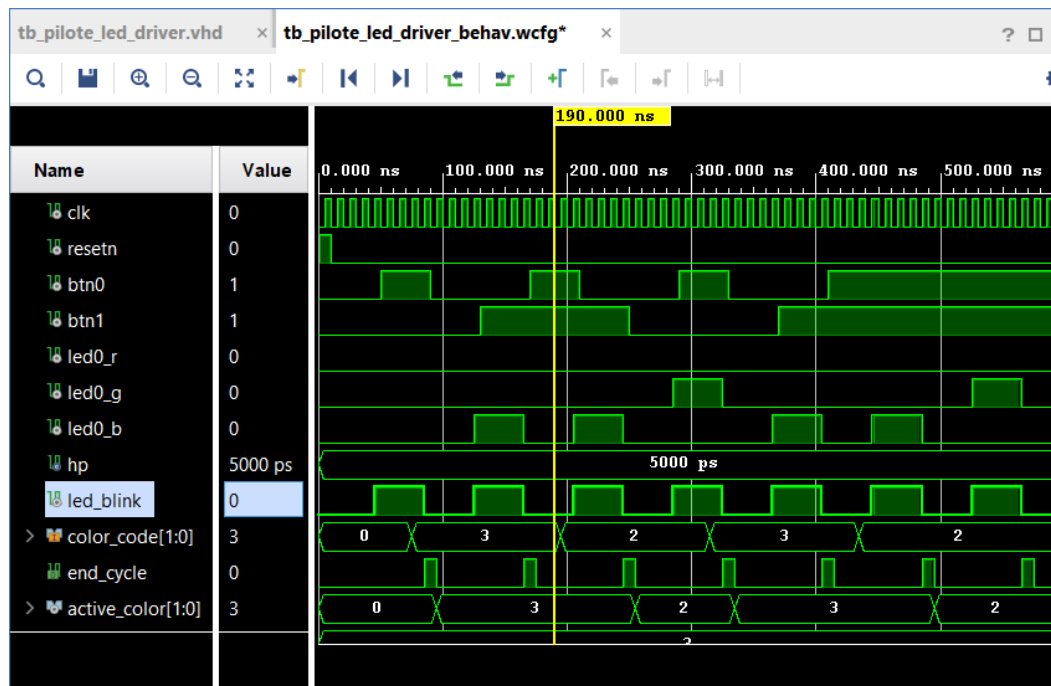
Voir code `pilote_led_driver`, `cycle_counter` et `LED_driver`

J'ai ajouté un port dans `cycle_counter` : `end_cycle`, calculé à partir de « cycle » et « not blink » puis je l'ai ajouté dans `cycle_counter` via le port map.

J'ai adapté mes codes suite à la création de la FIFO, le renommage du signal « update » de `pilote_led_driver` pour éviter les doublons et la confusion en faisait partie. J'ai également modifié l'instanciation du composant `LED_driver` dans `pilote_led_driver` : j'ai relié `color_code` à `dout` et `update` à `read_en` (`end_cycle` and `not empty`).

4. Mettez à jour le testbench et réalisez une simulation pour vérifier votre design.

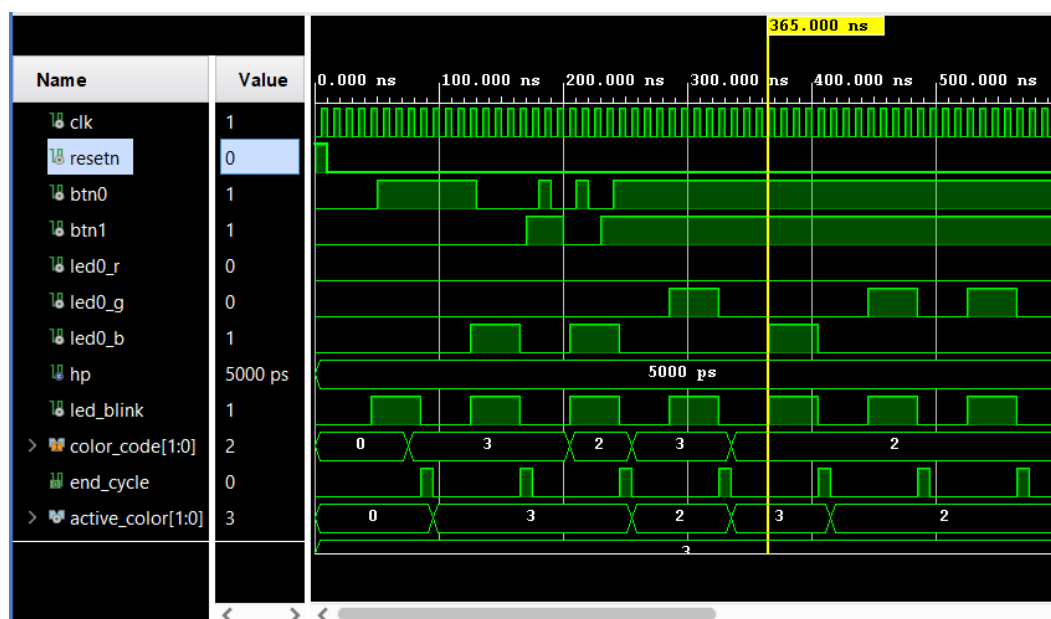
J'ai mis à jour le test bench de pilote_led_driver, afin qu'il puisse vérifier que le changement de couleur s'effectue correctement :



Mais dans un premier temps je ne comprenais pas bien le système de FIFO et de file d'attente, avec ma simulation j'avais l'impression que c'était l'appuie sur les boutons qui faisait le changement de couleur. Mais c'était dû au timing de led_blink et surtout au fait que je mettais trop de délais entre mes tests.

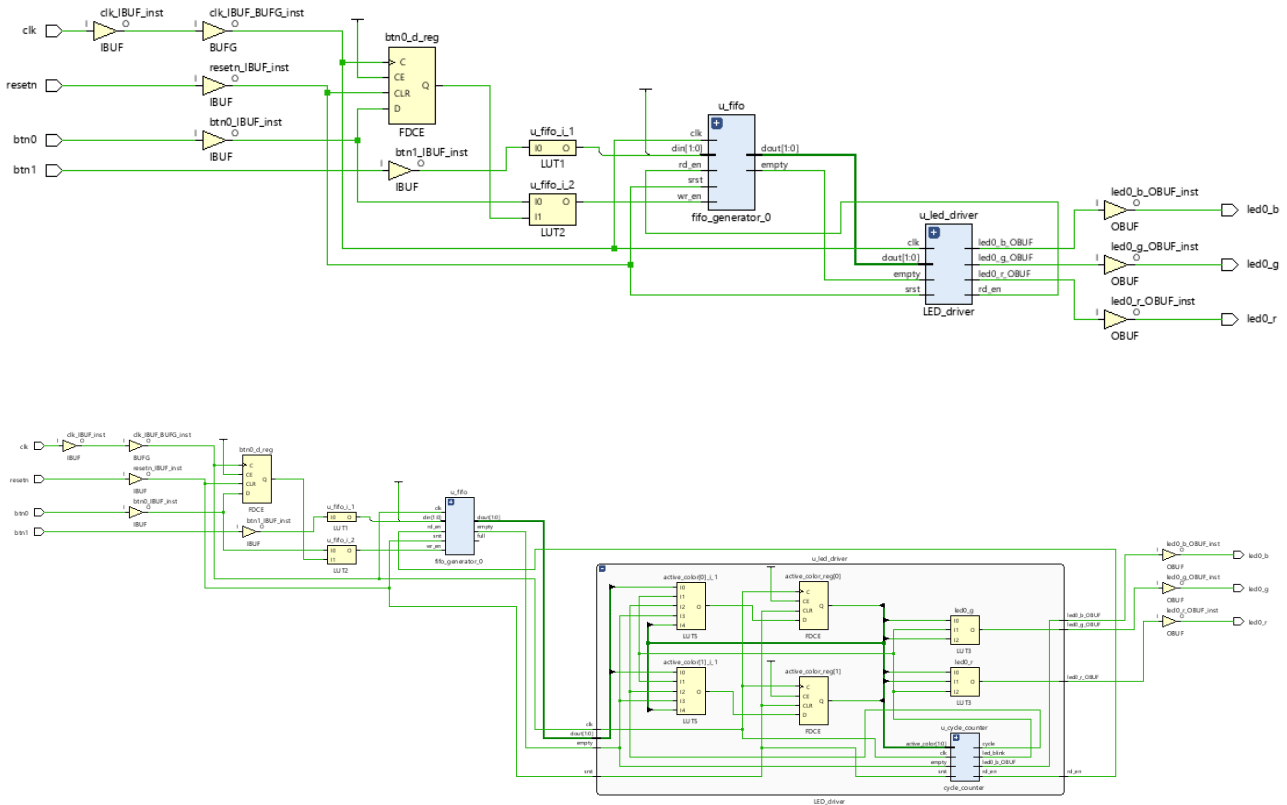
J'ai donc modifié mon test bench pour créer une séquence que la FIFO garderait en mémoire.

On peut voir qu'il y a une alternance avec les boutons 0 et 1 afin de demander un changement de couleur et de les sauvegarder dans la FIFO mais la couleur ne change pas tout de suite. Elle va d'abord terminer la période en cours puis changer de couleur pour faire alterner le bleu et le vert :



5. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

Voici la schématic du circuit on a bien 4 entrées et 3 sorties, je n'ai pas déplié la fifo car elle est immense et ça ne sert pas à grand-chose (je l'ai mise à la fin pour les curieux), on voit les 5 entrées (clk, din, rd_en, srst, wr_en) et les 2 sorties (dout et empty).



Le rapport de timing est cohérent, la marge est assez grande, et on a le message « All user specified timing constraints are met ».

Rapport de timing

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,956 ns	Worst Hold Slack (WHS): 0,144 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 104	Total Number of Endpoints: 104	Total Number of Endpoints: 70

All user specified timing constraints are met.

Le rapport d'utilisation qui nous donne le nombre de LUTs et de registre utilisé est cohérent : 28 registres pour counter_unit, le signal counter est déclaré sur 28 bits, soit un bit par registre.

Les 7 Bonded IOB correspondent aux entrée/sorties qui sont bien au nombre de 7.

La profondeur de la FIFO ne prends que 0,5 Block RAM, (32 c'est très petit comme file d'attente c'est donc normal).

Le BUFGCTRL correspond à l'horloge distribué à tous les blocks, il est égale à 1 car on utilise qu'une seule horloge pour tous les modules

Rapport d'utilisation

Name	Slice LUTs (17600)	Slice Registers (35200)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N pilote_led_driver	68	67	0.5	7	1
> I u_fifo (fifo_generator_0)	26	34	0.5	0	0
▼ I u_led_driver (LED_driver)	40	32	0	0	0
▼ I u_cycle_counter (cycle_c	37	30	0	0	0
I u_counter_unit (coun	36	28	0	0	0

6. Effectuez le placement routage et étudiez les rapports.

Suite au routage, le rapport de timing à légèrement évolué mais il reste très bon avec une bonne marge (slack) et on a encore le message « All user specified timing constraints are met ».

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,888 ns	Worst Hold Slack (WHS): 0,133 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 104	Total Number of Endpoints: 104	Total Number of Endpoints: 70
All user specified timing constraints are met.		

Le rapport d'utilisation a également un peu évolué mais on ne remarque pas de gros changement entre les rapports avant et après implémentation.

Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N pilote_led_driver	63	67	24	63	0.5	7	1
> I u_fifo (fifo_generator_0)	21	34	11	21	0.5	0	0
▼ I u_led_driver (LED_driver)	40	32	15	40	0	0	0
▼ I u_cycle_counter (cycle_c	37	30	14	37	0	0	0
I u_counter_unit (coun	36	28	14	36	0	0	0

7. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

Après avoir généré le bitstream et envoyé le code dans la carte CORA Z7, j'ai pu tester mon code avec les boutons de la carte.

La réaction des leds suit bien le fonctionnement attendu :

Les leds sont d'abord éteintes,

lorsque le bouton 0 est enfoncé la led bleue s'allume, et elle continue de clignoter indéfiniment.

Lorsque le bouton 1 est maintenu et que le bouton 0 subit une pression la couleur de la led passe en vert à la fin de la période en cours.

Et si je fais une séquence de changement de couleur rapide avec les boutons 0 et 1, la led alterne les couleurs bleu et vert en attendant la fin du cycle précédent à chaque fois.

Voir vidéo de démonstration

La schématique complètement dépliée du tp4_part2

